

Abhishek Raghav

Btech CSE

CU24250156

Experiment No. 5: Feature Extraction and Image Analysis using SIFT and HOG Descriptors

Aim

To implement Scale-Invariant Feature Transform (SIFT) and Histogram of Oriented Gradients (HOG) techniques for feature extraction and analyze their effectiveness in image representation, object recognition, image matching, and computer vision applications.

Requirements

- Python
- OpenCV (cv2)
- NumPy
- Matplotlib
- Scikit-image
- Jupyter Notebook / Google Colab / VS Code

1. Import Libraries and Load Image

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.feature import hog
from skimage import exposure

image = cv2.imread("/content/247b4f0f6feb39a745bcf6b16e24a76b.jpg")
if image is None:
    print("Error: Image not found. Please make sure 'image.jpg' is in the correct path.")
else:
    print("Image loaded successfully")

# Convert BGR to RGB for matplotlib display
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Display image
plt.figure(figsize=(8, 6))
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()
```

Image loaded successfully

Original Image



✓ 2. Convert Image to Grayscale

```
if image is not None:
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    plt.figure(figsize=(8, 6))
    plt.imshow(gray, cmap="gray")
    plt.title("Grayscale Image")
    plt.axis("off")
    plt.show()
else:
    print("Cannot convert to grayscale: Original image not loaded.")
```

Grayscale Image



✓ 3. SIFT Feature Extraction

SIFT is used to detect important points, called keypoints, in an image and calculate feature descriptors for those points. The experiment requires detecting SIFT keypoints and computing their descriptors.

```
if 'gray' in locals() and gray is not None:
    # Create SIFT object
    sift = cv2.SIFT_create()

    # Detect keypoints and calculate descriptors
    keypoints, descriptors = sift.detectAndCompute(gray, None)

    print("Number of keypoints:", len(keypoints))
    # Descriptors might be None if no keypoints are found or if the image is too small
```

```

if descriptors is not None:
    print("Descriptor shape:", descriptors.shape)
else:
    print("No descriptors found.")
else:
    print("Cannot perform SIFT: Grayscale image not available.")

```

```

Number of keypoints: 4556
Descriptor shape: (4556, 128)

```

Display SIFT Keypoints

```

if 'image_rgb' in locals() and image_rgb is not None and 'keypoints' in locals() and keypoints is not None:
    sift_image = cv2.drawKeypoints(
        image_rgb,
        keypoints,
        None,
        flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
    )

    plt.figure(figsize=(10, 7))
    plt.imshow(sift_image)
    plt.title("SIFT Keypoints")
    plt.axis("off")
    plt.show()
else:
    print("Cannot display SIFT keypoints: Original RGB image or keypoints not available.")

```

SIFT Keypoints



Observation

The SIFT algorithm detects important points such as corners, edges, and textured regions. Each keypoint is represented by a descriptor that contains information about its local image region.

4. HOG Feature Extraction

HOG stands for Histogram of Oriented Gradients. It represents an image using the distribution of gradient directions. The experiment specifies using parameters such as cell size, block size, and orientation bins.

```

if 'gray' in locals() and gray is not None:
    # Convert grayscale image to float (required by scikit-image's HOG)
    gray_float = gray.astype("float32") / 255.0

    # HOG feature extraction
    features, hog_image = hog(

```

```

        gray_float,
        orientations=9,
        pixels_per_cell=(8, 8),
        cells_per_block=(2, 2),
        block_norm="L2-Hys", # L2-Hys is a common normalization method
        visualize=True,
        channel_axis=None # Specify no channel axis for grayscale image
    )

    print("Number of HOG features:", len(features))
else:
    print("Cannot perform HOG: Grayscale image not available.")

```

Number of HOG features: 524160

▼ Display HOG Image

```

if 'hog_image' in locals() and hog_image is not None:
    # Rescale HOG image for better visualization
    # The in_range parameter might need adjustment depending on the image contrast
    hog_image_rescaled = exposure.rescale_intensity(
        hog_image,
        in_range=(0, 1.0) # HOG image values are typically between 0 and 1
    )

    plt.figure(figsize=(10, 7))
    plt.imshow(hog_image_rescaled, cmap="gray")
    plt.title("HOG Visualization")
    plt.axis("off")
    plt.show()
else:
    print("Cannot display HOG image: HOG features not computed.")

print("The HOG visualization shows the gradient orientation patterns captured from the image.")

```

HOG Visualization



The HOG visualization shows the gradient orientation patterns captured from the image.

5. Comparison of SIFT and HOG

Feature	SIFT	HOG
Full Form	Scale-Invariant Feature Transform	Histogram of Oriented Gradients
Main purpose	Local feature detection and description	Shape and edge representation
Output	Keypoints + descriptors	Feature vector
Scale invariant	Yes	No, generally
Rotation invariant	Yes	Limited
Main information	Local texture and structure	Gradient/edge information

Feature	SIFT	HOG
Computational complexity	Relatively higher	Generally lower
Best suited for	Image matching, object recognition	Object detection, shape recognition
Feature type	Local	Dense/global-region based

The experiment specifically asks comparison based on scale invariance, rotation invariance, computational efficiency, robustness, and applications.

6. Image Matching using SIFT

We can compare two similar images using SIFT descriptors. For this section, you'll need `image1.jpg` and `image2.jpg` in your environment.

```
# Load images (assuming 'image1.jpg' and 'image2.jpg' are available)
image1 = cv2.imread("/content/247b4f0f6feb39a745bcf6b16e24a76b.jpg", cv2.IMREAD_GRAYSCALE)
image2 = cv2.imread("/content/247b4f0f6feb39a745bcf6b16e24a76b.jpg", cv2.IMREAD_GRAYSCALE)

if image1 is None or image2 is None:
    print("Error: One or both images for matching (image1.jpg, image2.jpg) not found.")
else:
    sift = cv2.SIFT_create()

    kp1, des1 = sift.detectAndCompute(image1, None)
    kp2, des2 = sift.detectAndCompute(image2, None)

    if des1 is not None and des2 is not None:
        # FLANN based matcher parameters
        FLANN_INDEX_KMEANS = 1 # For SIFT/SURF
        index_params = dict(algorithm = FLANN_INDEX_KMEANS, trees = 5)
        search_params = dict(checks = 50)

        flann = cv2.FlannBasedMatcher(index_params, search_params)

        matches = flann.knnMatch(des1, des2, k=2)

        # Lowe's ratio test to filter good matches
        good_matches = []
        for m, n in matches:
            if m.distance < 0.7 * n.distance:
                good_matches.append(m)

        print("Total good matches:", len(good_matches))
    else:
        print("No descriptors found for one or both images, cannot perform matching.")
```

Total good matches: 4654

Display Matches

```
if 'image1' in locals() and image1 is not None and \
    'image2' in locals() and image2 is not None and \
    'kp1' in locals() and kp1 is not None and \
    'kp2' in locals() and kp2 is not None and \
    'good_matches' in locals() and good_matches is not None:

    result = cv2.drawMatches(
        image1, kp1,
        image2, kp2,
        good_matches, None,
        flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
    )

    plt.figure(figsize=(15, 8))
    plt.imshow(result, cmap="gray")
    plt.title("SIFT Image Matching")
    plt.axis("off")
    plt.show()
else:
    print("Cannot display matches: Images, keypoints, or good matches not available.")

print("This implements the experiment's requirement of applying SIFT/HOG features for basic image matching or object recognition")
```

SIFT Image Matching



This implements the experiment's requirement of applying SIFT/HOG features for basic image matching or object recognition using

Questions and Answers

Q1. What is feature extraction, and why is it important in computer vision?

Answer:

Feature extraction is the process of identifying and representing important information from an image in the form of numerical features or descriptors.

Instead of processing every pixel directly, feature extraction identifies useful properties such as:

- Edges
- Corners
- Shapes
- Textures
- Gradient information
- Local patterns

Feature extraction is important because it reduces the complexity of image data and provides meaningful information for image matching, object recognition, tracking, classification, and detection.

Q2. Explain the working principle of SIFT.

Answer:

SIFT stands for Scale-Invariant Feature Transform. It extracts distinctive local features from an image.

The main steps are:

- **Scale-space construction** – The image is processed at different scales.
- **Keypoint detection** – Important points are identified using differences of Gaussian images.
- **Keypoint localization** – Unstable points are removed.
- **Orientation assignment** – A dominant orientation is assigned to each keypoint.
- **Descriptor generation** – A numerical descriptor is created around each keypoint.
- **Feature matching** – Descriptors from different images can be compared to find corresponding features.

SIFT is useful for image matching and object recognition because its features are robust to changes in scale and rotation.

Q3. What are keypoints and feature descriptors?

Answer:

Keypoints are important locations in an image where significant visual information exists. They can occur at corners, edges, or textured regions.

A **feature descriptor** is a numerical representation of the region surrounding a keypoint.

For example, in SIFT:

- Keypoint → location of an important feature
- Descriptor → numerical information describing that feature

Descriptors can then be compared between images to identify matching regions.

Q4. Explain HOG and its significance.

Answer:

HOG stands for Histogram of Oriented Gradients.

HOG describes an image by analyzing the direction and strength of gradients.

The basic process is:

- Divide the image into small cells.
- Calculate image gradients.
- Find the orientation of gradients.
- Create a histogram of gradient orientations.
- Group cells into blocks.
- Normalize the histograms.
- Combine them into a feature vector.

HOG is particularly useful for representing object shapes and edges and has been widely used for object detection.

Q5. Compare SIFT and HOG based on robustness, computational complexity, and applications.

Answer:

SIFT

- More robust to scale changes.
- Rotation invariant.
- Provides distinctive local descriptors.
- Computationally more expensive.
- Useful for image matching and object recognition.

HOG

- Represents edge and shape information.
- Computationally simpler than SIFT in many applications.
- Not inherently scale invariant.
- Has limited rotation robustness.
- Commonly useful for object detection and shape representation.

Therefore, SIFT is better for local feature matching, while HOG is suitable for shape and edge-based object detection.

Q6. Why is SIFT considered invariant to scale and rotation?

Answer:

SIFT is **scale invariant** because it searches for keypoints across multiple image scales using a scale-space representation.

It is **rotation invariant** because SIFT assigns a dominant orientation to each keypoint. The descriptor is then constructed relative to this orientation.

Therefore, when an image is resized or rotated, important features can still be detected and matched.

Q7. Mention three real-world applications of HOG.

Answer:

Three applications of HOG descriptors are:

1. Pedestrian detection
2. Vehicle/object detection

3. Human shape recognition

HOG is useful in these applications because it effectively represents the edges and shape of objects.

Q8. Why is feature extraction performed before image classification or object detection?

Answer:

Raw images contain a large number of pixels and unnecessary information. Processing all pixels directly can be computationally expensive.

Feature extraction converts the raw image into a smaller and more meaningful representation.

For example:

Raw Image → Feature Extraction → Feature Vector → Classifier → Result

This helps the classification or detection system focus on important visual characteristics such as shape, edges, texture, and local patterns.

Q9. What are the advantages and limitations of handcrafted feature descriptors compared to deep learning?

Answer:

Advantages (Handcrafted Features like SIFT/HOG)

- Easy to understand.
- Usually requires less training data.
- Computationally less demanding during training.
- Useful for specific types of visual features.
- Can work effectively on small datasets.

Limitations (Handcrafted Features)

- Features are manually designed.
- May not perform well for highly complex images.
- Less adaptable to different environments.

Deep learning methods can learn more complex representations automatically. Deep learning methods learn features directly from training data, while SIFT and HOG are handcrafted feature extraction techniques.

Q10. How do feature extraction techniques contribute to image matching, face recognition, and object detection?

Answer:

Feature extraction provides a numerical representation of important visual information.

- **Image Matching** SIFT descriptors can be extracted from two images and compared to find corresponding points.
- **Face Recognition** Features representing important facial structures can be extracted and compared between faces.
- **Object Detection** HOG can represent the shape and edge structure of objects, which can then be used by a classifier to identify objects.

Thus, feature extraction acts as an important intermediate step between the raw image and the final computer vision task. The experiment identifies image matching, recognition, tracking, and classification as key uses of robust feature descriptors.

✓ Result

Result: SIFT and HOG feature extraction techniques were successfully implemented. SIFT was used to detect keypoints and generate feature descriptors, while HOG was used to represent gradient orientation and shape information. Their characteristics, robustness, computational requirements, and applications were compared.